ИТМО

Алгоритмы и структуры данных

Лекция 1

Виды деревьев и графов

И о втором семестре поговорим

Сначала графы, потом деревья

ИТМО

Работа с графами	8	8	-	-	9.4

Тема	Ссылка
Введение в курс	
Виды деревьев и графов	
BFS. DFS. Алгоритм Дейкстры	
Эвристические и оптимизированные методы обхода графа	

Работа с деревьями	8	8	-	-	9.4

Тема	Ссылка
Непересекающихся множеств и остовные деревья	
Бинарные деревья; красно-черные деревья	
B-деревья	
Расширение структур данных	

Нам еще куда-то **амортизационку** впихнуть надо…

Теоретические структуры данных

ИТМО

Теоритические алгоритмы и структуры данных	8	8	-	-	9.4

Тема	Ссылка
Задача о максимальном потоке	
Фибоначчиевы пирамиды	
Деревья ван Эмде Боаса	
Сливающееся дерево	

Они правда классные…

Ну правда…

Теоретические структуры данных

ИТМО

BLAS алгоритмы	8	8	-	-	9.4

Тема	Ссылка
Numpy	
Линейное программирование	
Вычислительная геометрия	
Полиномы и быстрое преобразование Фурье	

И напоследок повеселимся!

ПЛАН (примерный) по лабам

ИТМО

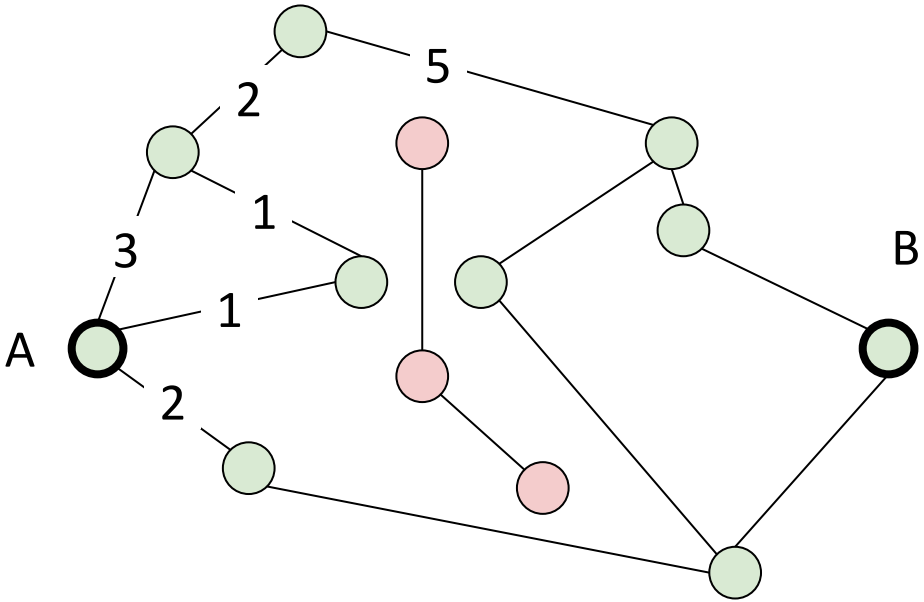
Тема	Комментарий	БАРС базовый	БАРС за раздел
ДП	долг	5	17
Жадники	долг	5	
Графы	Дейкстра, BFS, DFS и A* на хард	10	
КЧ и B деревья	Смотрим как logN влияет на разные операции	11	17
Непересекающиеся множества	Почти почти *O(1)	8	
Задача о максимальном потоке*	Н*ркотрип. Сегментируем через эту штуковину	9	16
Расширение структур данных	Адаптируем структуры данных под задачи	9	
Numpy	Тут ЧИЛ	3	10
Вычислительная геометрия	Помните геометрию с 8-го класса?	3	
Линейное программирование	Тут будем считать как торговать на WB выгоднее	4	

Задача для зазнаек

Реализовать A* для двух точек A и B для несвязного взвешенного графа

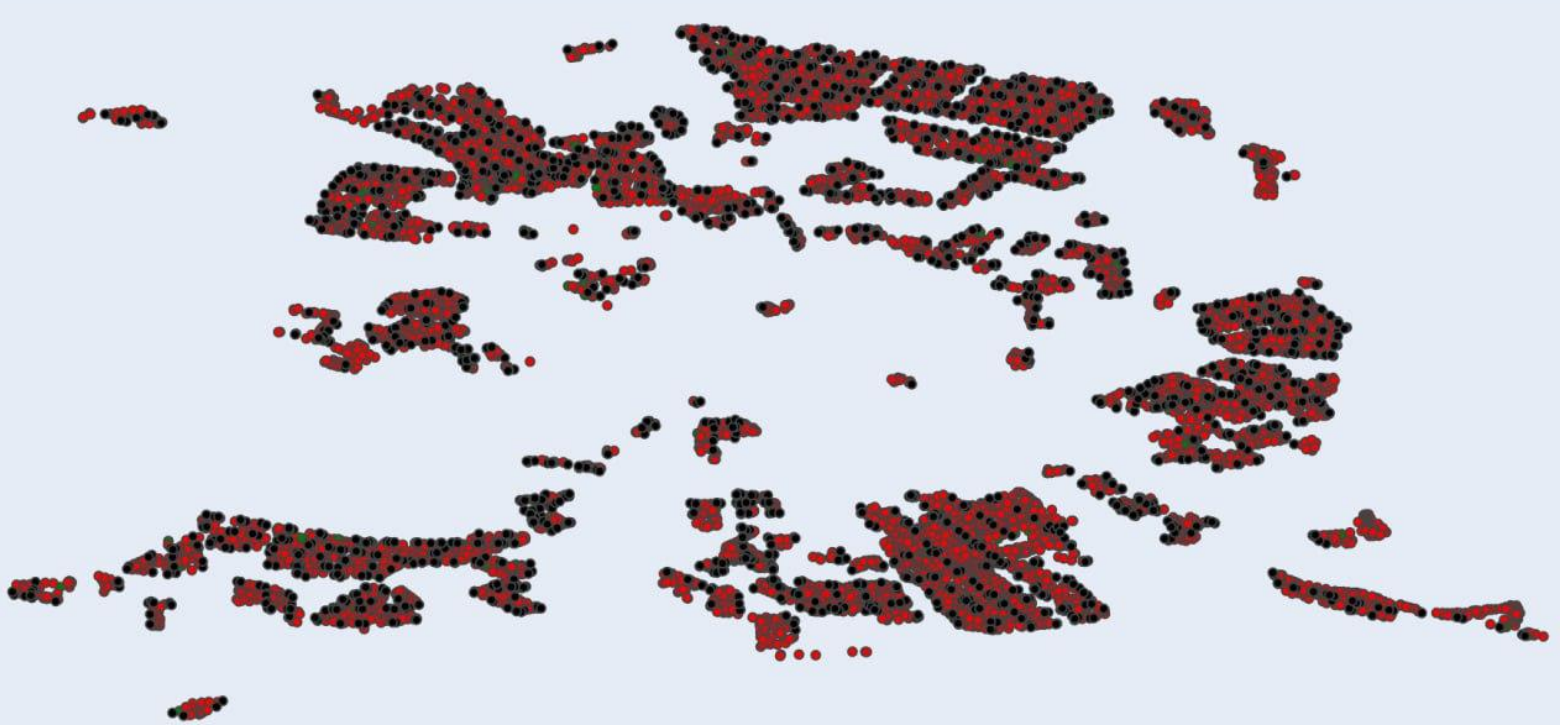
Поинт:

- Есть 2 подмножества вершин:
  - Узлы дорог
  - Узлы препятствия
- Нужная эвристика которая будет учитывать наличие препятствий



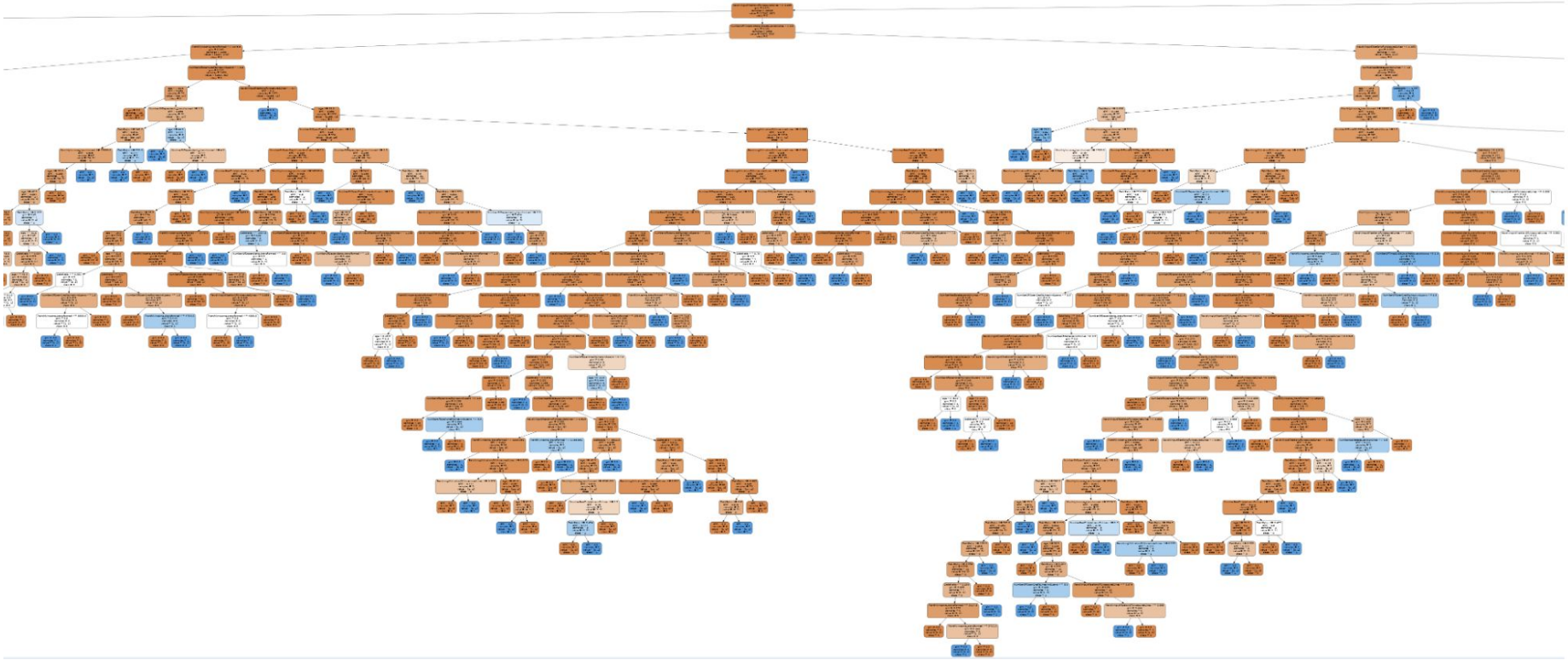
Графы - это красиво

ИТМО



А деревья красивы?

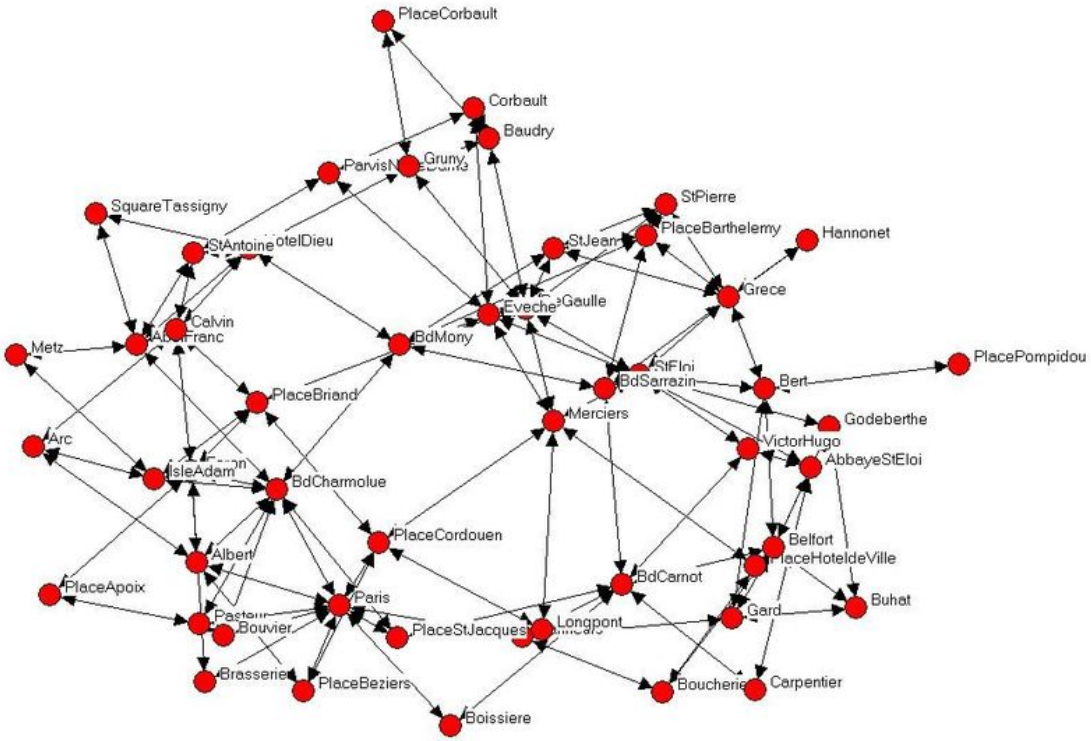
Фрагмент дерева «Decision Tree» на «Give Me Some Credit»



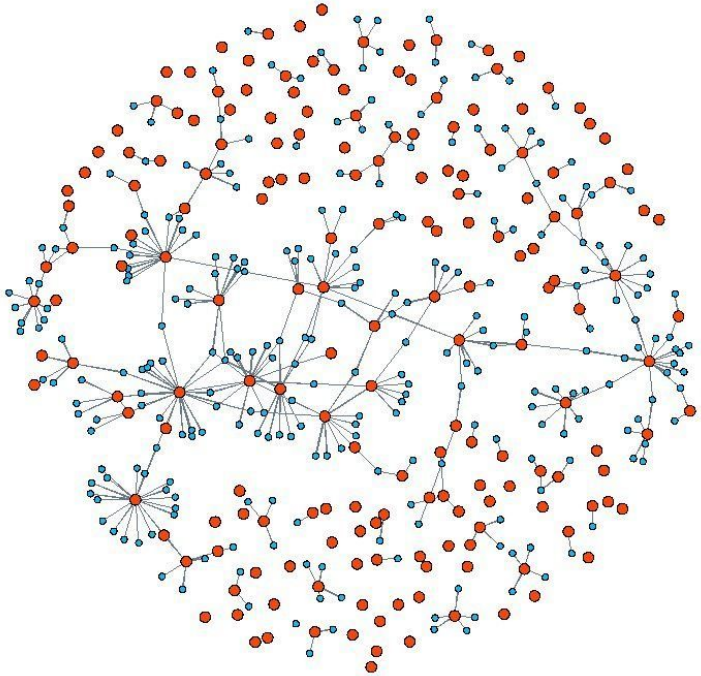
Где графы нужны в реальной жизни

Спойлер: везде

Граф реальной жизни / города



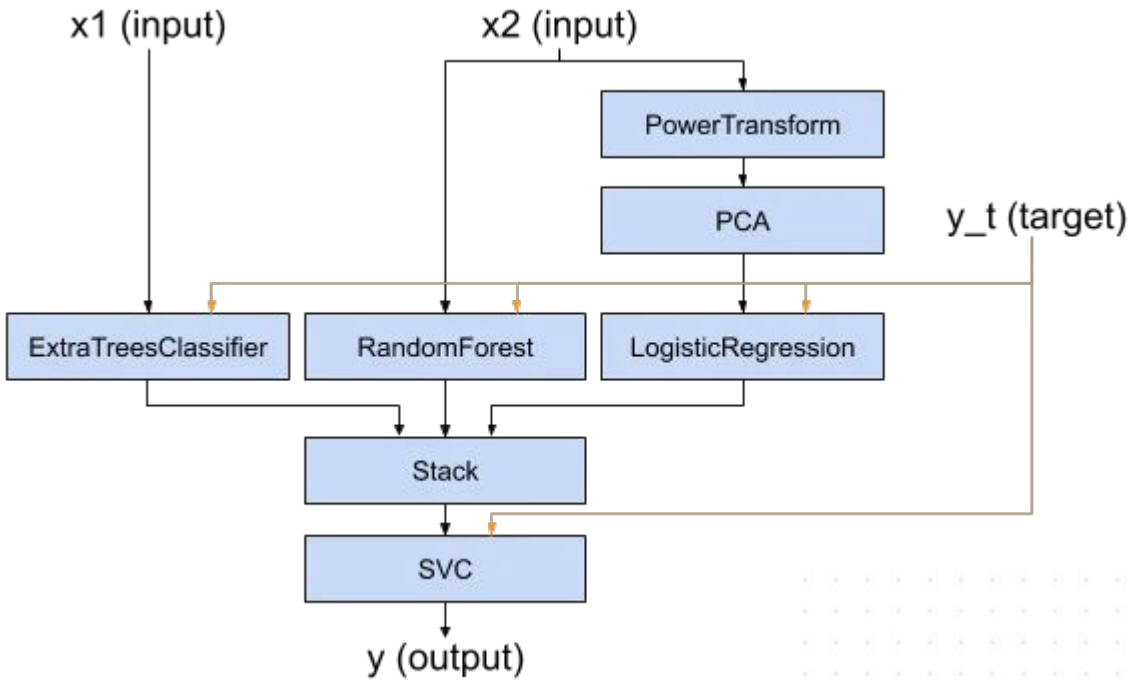
Граф реальной жизни / соц. сети



Граф реальной жизни / заметки

Obsidian

Граф реальной жизни / Pipelines

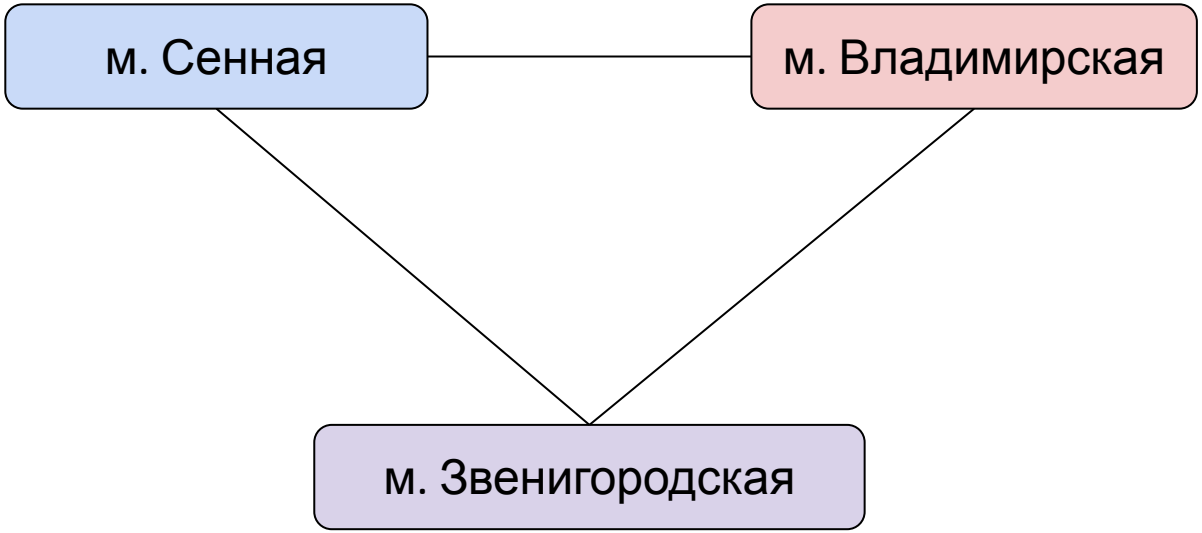


airflow

Го опять вернемся к графам

Какие существуют виды графов?

Выборка метро спб

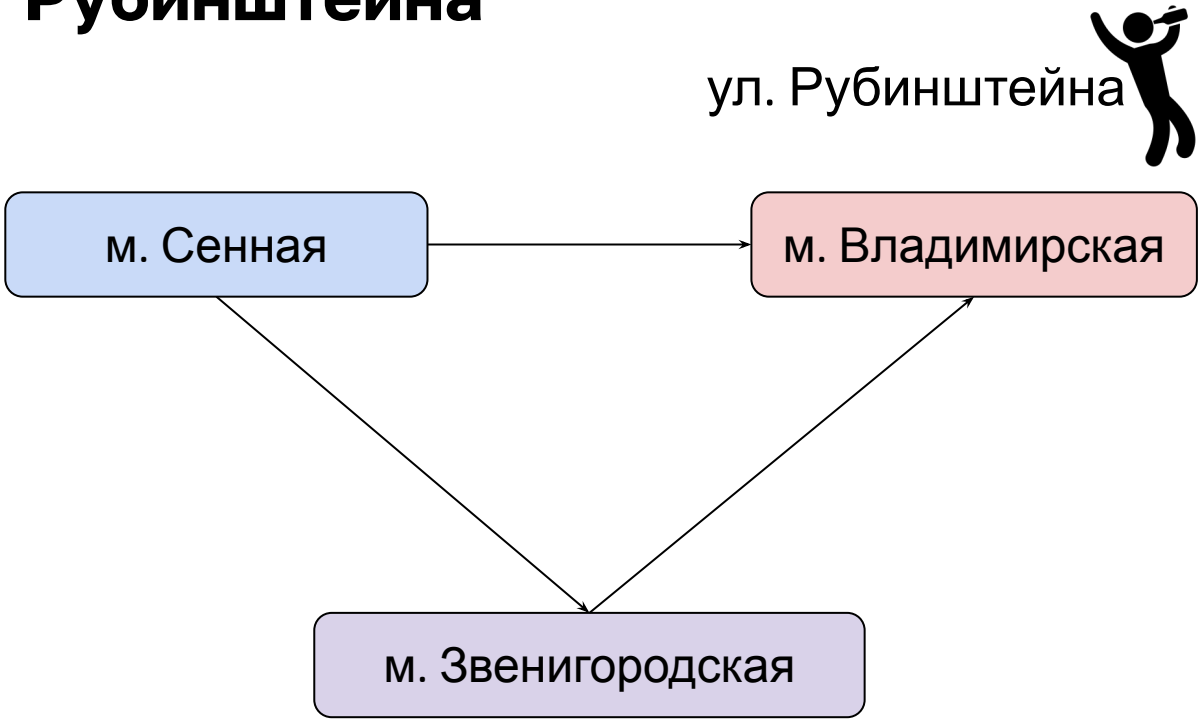


Какой граф?

Неориентированный граф

В пятницу все дороги ведут на Рубинштейна

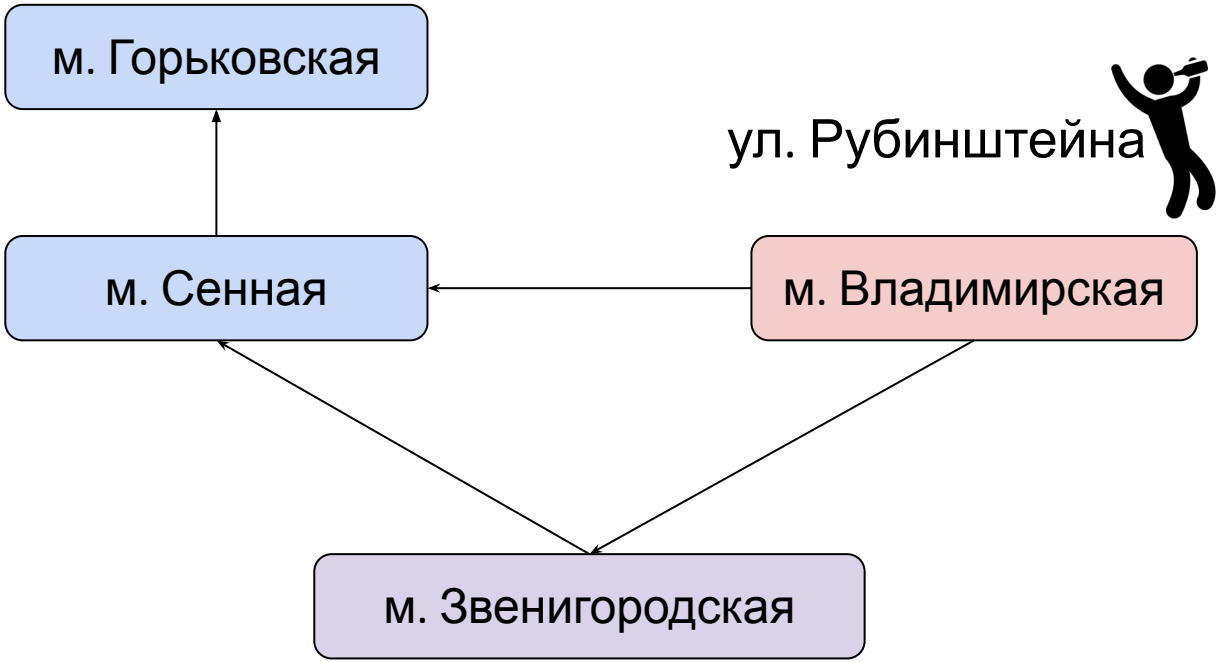
ИТМО



Какой граф?

Ориентированный граф

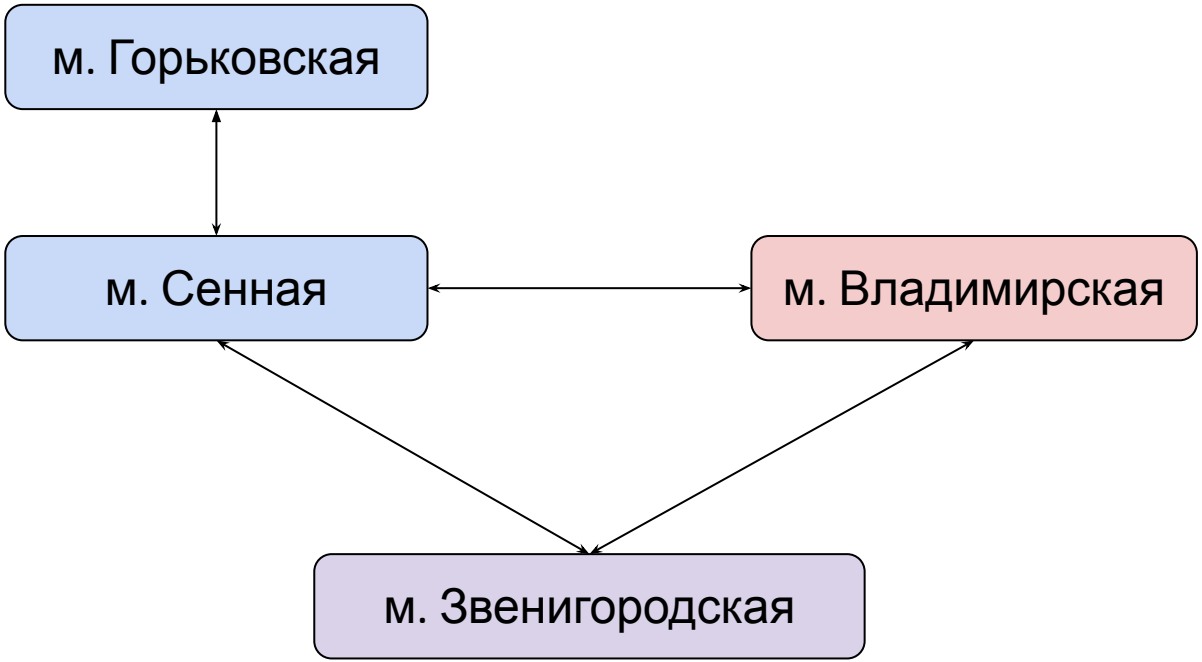
В пн все дороги ведут на лекцию по алгосам



Какой граф?

Ориентированный граф

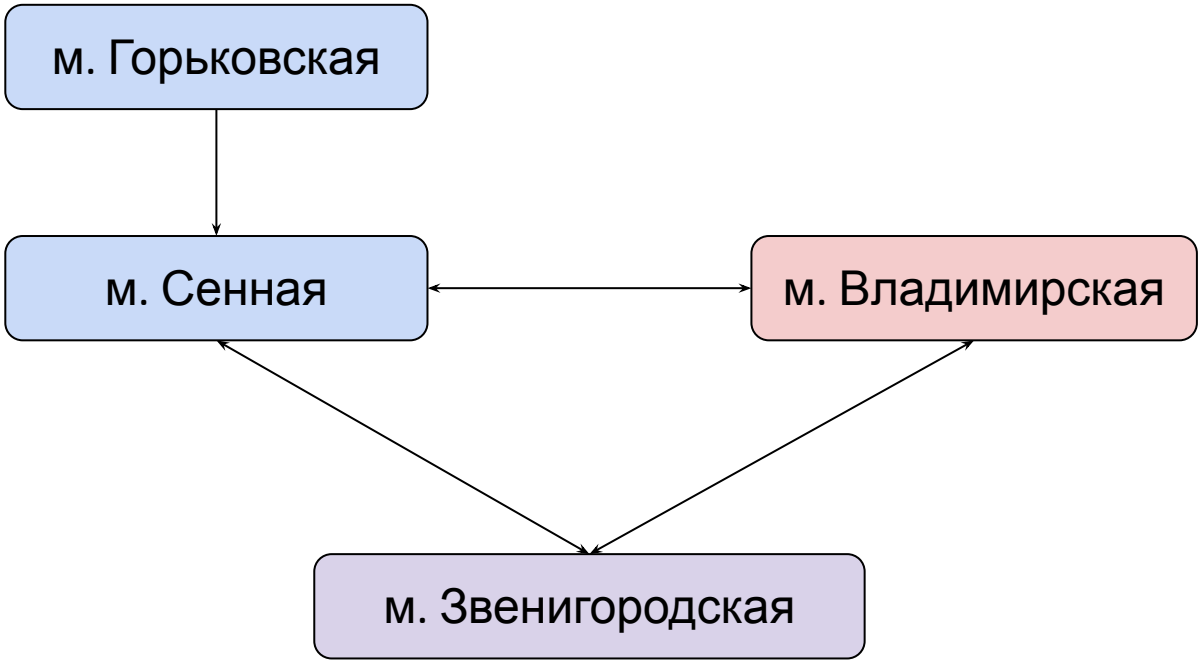
Когда нет алгосов и пятницы все дороги открыты



Какой граф?

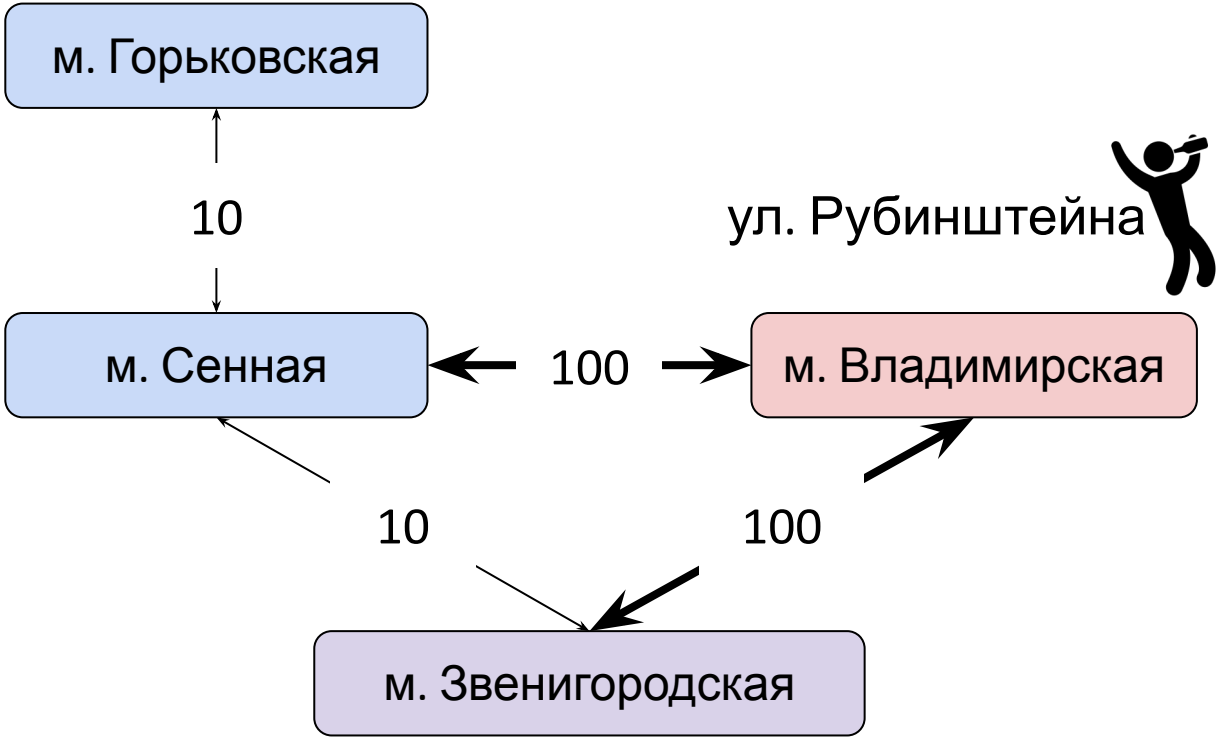
Двунаправленный граф

Хотя двунаправленный, скорее, когда точно не надо на пары…



Двунаправленный граф

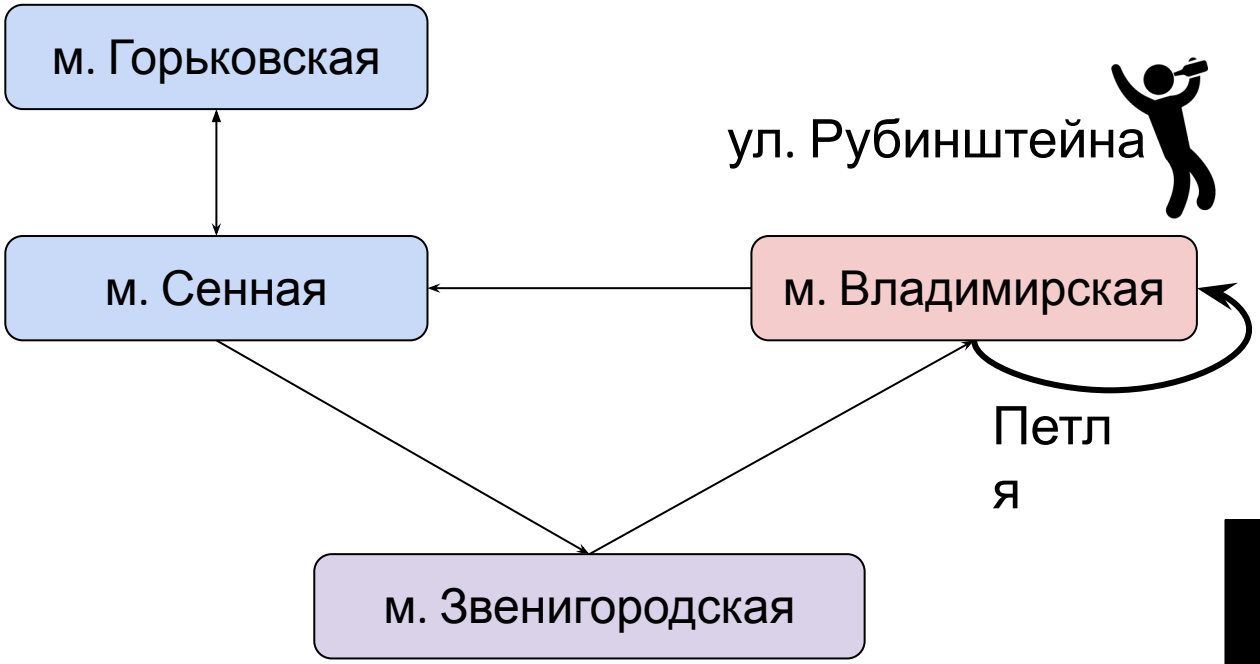
Когда коллоквиум в субботу



Какой граф?

Взвешенный граф

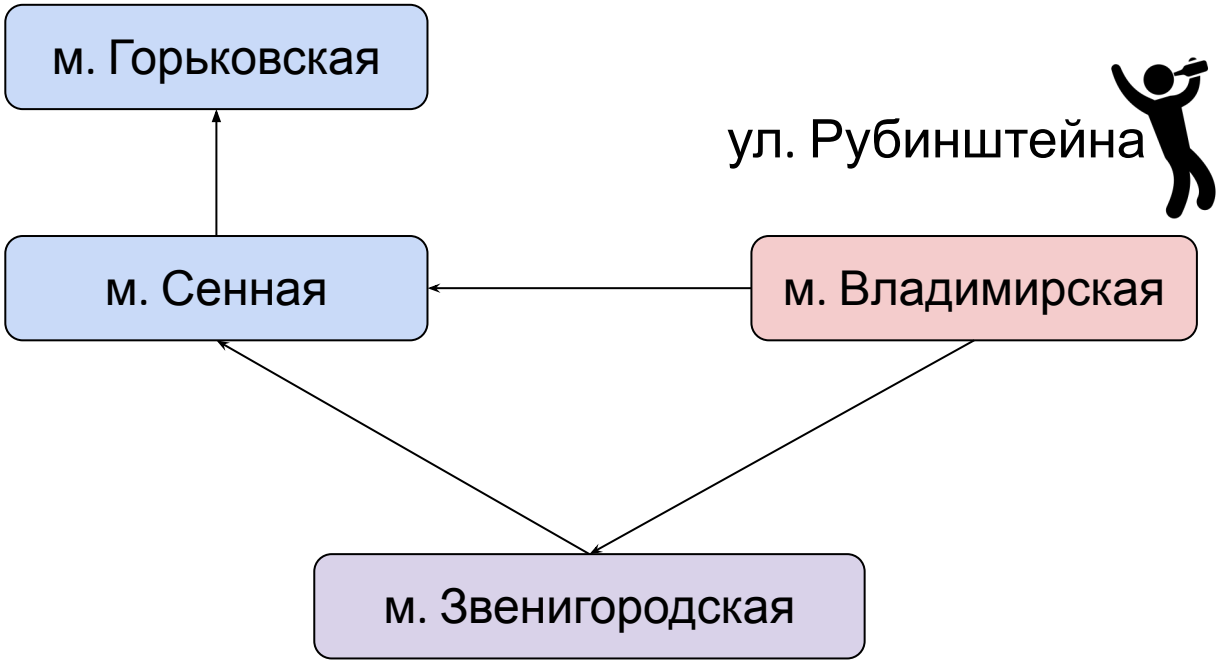
А когда рубика не отпускает?



Какой граф?

Циклический граф

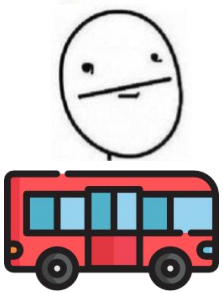
В каком графе тогда можно выйти из цикла?



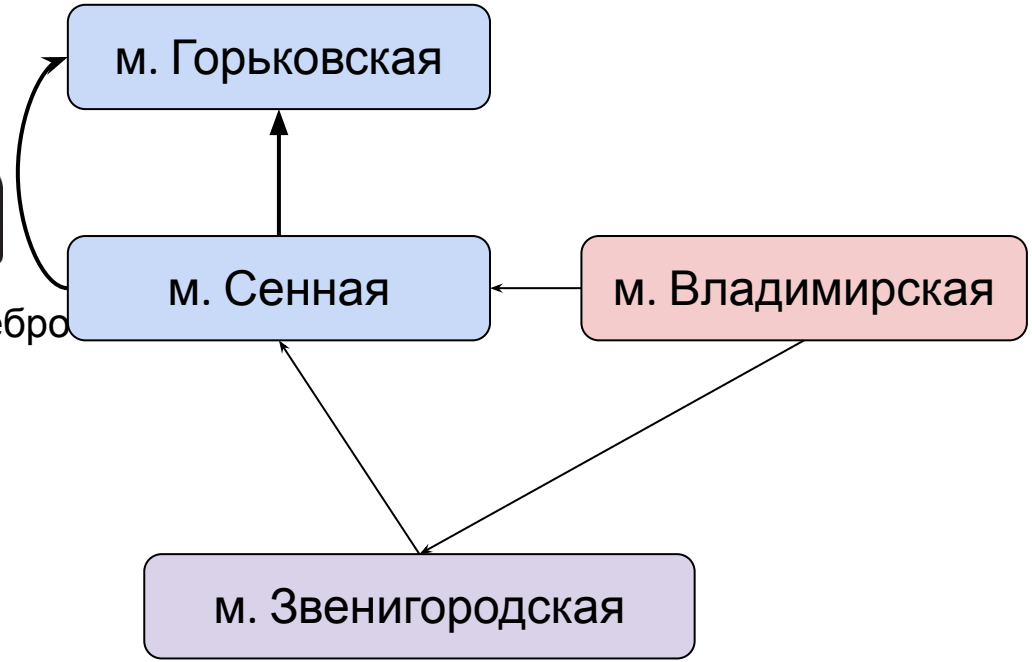
Какой граф?

Ациклический граф

Либо на метро, либо на автобусе



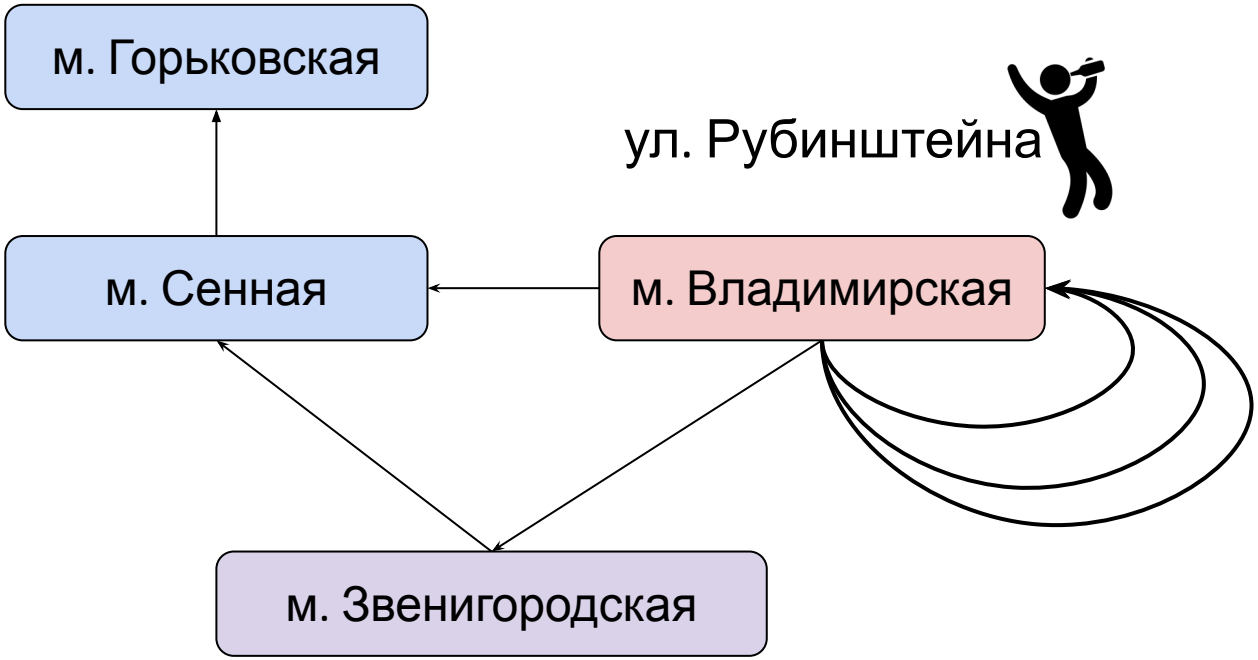
Кратное ребро



Какой граф?

Псевдограф

Псевдограф может быть и в субботу вечером



Еще есть полный граф

Когда все вершины соединены ребрами

Еще есть набор скучных названий

ИТМО

- **Связный граф** - если для любых вершин *u* и *v*, есть путь *u* в *v*
- **Сильно связным (ориентированно связным)** - если граф связный и из любой вершины есть путь в любую другую вершины
- **Двудольный** - когда есть 2 непересекающихся множества V1 и V2
- **k-дольный** - когда есть k непересекающихся множества V1, V2, Vk
- **Полный двудольный** - комбинация полного и двудольного
- **Планарный** - если граф можно отобразить на плоскости без пересекающихся ребер
- **Хордальный** - если граф не содержит индуцированных циклов с длиной больше трёх
- **Изоморфный** - как изоморфные строки, только графы
- **Смешанный** - ориентированный и неориентированный граф в одном месте
- **Мультиграф** - частный случай псевдографа без петель
- **Псевдомультиграф** - псевдограф с ограничением кратных ребер

Деревья

Дерево - это связный ациклический граф
Может быть ориентированным и не ориентированным

Связный - наличие пути между каждой парой вершин
Ациклический - нет циклов и петель

Форматы представления графа

ИТМО

Список смежности (adjacency list)

```
A: B, C
B: A, D
C: A
D: B
```

Матрица смежности (adjacency matrix)

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	0
D	0	1	0	0

Список ребер (edge list)

```
(A, B)
(A, C)
(B, D)
```

Форматы представления графа 2

ИТМО

GraphML

```
<graphml>
  <graph id="G">
    <node id="A"/>
    <node id="B"/>
    <edge source="A" target="B"/>
  </graph>
</graphml>
```

DOT (graphviz)

```
digraph G {
  A -> B;
  A -> C;
  B -> D;
}
```

Graph Modelling Language

```
graph [
  node [ id 1 label "A" ]
  node [ id 2 label "B" ]
  edge [ source 1 target 2 ]
]
```

Давайте ближе к коду

Создаем структуру узла графа

ИТМО

```
// Определяем структуру для узла графа
typedef struct {
  int node_id; // Данные узла (может быть ID или любое другое значение)
  Vector* neighbors;
} Node;
```

Создаем структуру графа

ИТМО

```
// Определяем структуру для узла графа
typedef struct {
  int node_id; // Данные узла (может быть ID)
  Vector* neighbors;
} Node;

typedef struct {
   Vector* nodes; // Вектор указателей на узлы графа
} Graph;
```

Какой это граф?

Создание графа

ИТМО

```
Graph* createGraph() {
    Graph* graph = malloc(sizeof(Graph));
    if (graph == NULL) {
        printf("Не удалось выделить память под граф \n");
        return NULL;
    }

    graph->nodes = createVector(sizeof(Node));

    return graph;
}
```

Добавление узла

ИТМО

```
int appendNode(Graph* graph, int node_id) {
    Node* node = malloc(sizeof(Node));
    if (node == NULL) {
        printf("Не удалось выделить память под узел \n");
        return -1;
    }

    node->node_id = node_id;
    node->neighbors = createVector(sizeof(Node));

    if (appendVectorItem(graph->nodes, node) != 0) {
        printf("Не удалось добавить узел в граф \n");
        return -1;
    }

    return 0;
}
```

Метод добавления ребра

ИТМО

```
int appendEdge(Graph* graph, int node_id_from, int node_id_to) {
    Node* node_from = findNode(graph, node_id_from);
    Node* node_to = findNode(graph, node_id_to);

    if (node_from == NULL || node_to == NULL) {
        printf("Не найден один из узлов по id \n");
        return -1;
    }

    if (appendVectorItem(node_from->neighbors, node_to) != 0) {
        printf("Не удалось добавить узел как ребро в узел \n");
        return -1;
    }

    return 0;
}
```

Тесты тесты тестики

ИТМО

```
int main() {
    Graph* graph = createGraph();
    assert(graph != NULL);

    assert(appendNode(graph, 1) == 0);
    assert(findNode(graph, 1)->node_id == 1);
    assert(graph->nodes->size == 1);

    assert(appendNode(graph, 2) == 0);
    assert(findNode(graph, 2)->node_id == 2);
    assert(graph->nodes->size == 2);

    assert(appendEdge(graph, 1, 2) == 0);
    assert(findNode(graph, 1)->neighbors->size == 1);
    assert(findNode(graph, 2)->neighbors->size == 0);

    printf("ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО");
    return 0;
}
```

Мне кажется

Или у вас есть вопросы?

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru